Open in Colab

Algoritmos de optimización - Seminario

Nombre y Apellidos: Ángel Encinas Moreno & Williams Mérida Parard
Url:https://github.com/Williams-MP/Algoritmos_de_optimizacion_entregas/tree/9086b65de7cbc459251d3467e8c
Problema:

> 1. Sesiones de doblaje

Descripción del problema:

Se precisa coordinar el doblaje de una película. Los actores del doblaje deben coincidir en las tomas en las que sus personajes aparecen juntos en las diferentes tomas. Los actores de doblaje cobran todos la misma cantidad por cada día que deben desplazarse hasta el estudio de grabación independientemente del número de tomas que se graben. No es posible grabar más de 6 tomas por día. El objetivo es planificar las sesiones por día de manera que el gasto por los servicios de los actores de doblaje sea el menor posible. Los datos son:

Número de actores: 10 Número de tomas : 30 Actores/Tomas : https://bit.ly/36D8IuK

(*) La respuesta es obligatoria

Uso de la IA

Para la obtención de los bloques de código nos hemos apoyado en IA generativa, tanto para optimizar ideas propias como para solucionar errores persistentes que encontrábamos en el camino. En ningún momento se ha generado desde cero y simplemente copiado y pegado ningún bloque de código.

Importación de librerias necesarias

In [1]:
```
import pandas as pd
import numpy as np
import time
import math
import random
import itertools
```

Carga de datos orginales

In [2]:
```
# Cargamos el archivo excel en la misma ruta.
nombre_archivo = "Datos problema doblaje(30 tomas, 10 actores).xlsx
# Ajusta el skiprows o usecols si tu Excel tiene cabeceras raras
df_completo = pd.read_excel(nombre_archivo)

# Seleccionar los datos numéricos y evitamos las cabeceras
datos_numericos = df_completo.iloc[1:31, 1:11].dropna().astype(int)

# Creamos un dataframe con los datos
registro_sesiones = pd.DataFrame(
    datos_numericos.values,
    index=np.arange(len(datos_numericos)),
    columns=['Actor ' + str(x) for x in np.arange(1, 11)]
)
registro_sesiones.index.name = 'Toma -1 '

# Calculamos el total, con esto aseguramos cualquier discrepancia c
registro_sesiones['Total'] = np.sum(registro_sesiones, axis=1)

# Visualizamos los resultados
display(registro_sesiones.head())

# Asignamos el DF a la variable data.
data = registro_sesiones
```

Toma -1	Actor 1	Actor 2	Actor 3	Actor 4	Actor 5	Actor 6	Actor 7	Actor 8	Actor 9	Actor 10
0	1	1	1	1	1	0	0	0	0	0
1	0	0	1	1	1	0	0	0	0	0
2	0	1	0	0	1	0	1	0	0	0
3	1	1	0	0	0	0	1	1	0	0
4	0	1	0	1	0	0	0	1	0	0

Dataframe con valores aleatorios

Más adelante se solicitará la creación de una muestra de datos aleatorias. En nuestro caso hemos decidido, crearla ahora, para poder ser utilizada en todos los algoritmos presentados en este proyecto. Los valores aleatorios siguen una distribución uniforme, que lo se verá reflejado en el coste cuando se muestren los resultados.

In [3]:
```
def generar_datos_aleatorios_uniformes(n_tomas, n_actores):
```

```
    """
    Genera una matriz de ceros y unos totalmente aleatoria.
    """
    # Crear matriz de n_tomas x n_actores con 0 y 1
    data = np.random.randint(2, size=(n_tomas, n_actores), dtype='i

    df = pd.DataFrame(data,
                      index=np.arange(len(data)),
                      columns=['Actor ' + str(x) for x in np.arange

    df.index.name = 'Toma -1'

    # Añadimos la columna 'Total' que suelen usar los algoritmos vo
    df['Total'] = df.sum(axis=1)

    return df

# EJEMPLO DE USO:
# n_tomas = 20, n_actores = 8
df_aleatorio = generar_datos_aleatorios_uniformes(30, 10)
print("Nuevo set de datos generado aleatorios con distribuación uni
display(df_aleatorio.head())
```

```
Nuevo set de datos generado aleatorios con distribuación uniforme:
```

Toma -1	Actor 1	Actor 2	Actor 3	Actor 4	Actor 5	Actor 6	Actor 7	Actor 8	Actor 9	Actor 10
0	1	1	0	1	1	1	0	1	1	0
1	1	0	1	0	1	0	1	1	0	1
2	0	1	1	0	0	1	1	0	0	0
3	0	0	0	0	0	0	1	1	0	1
4	0	1	1	1	1	1	0	0	0	0

¿Cuantas posibilidades hay sin tener en cuenta las restricciones?

Si no tenemos en cuenta las restricciones de capacidad y tiempo se podría asegurar que las posibilidades serían virtualmente infinitas. Al no existir un número mínimo de tomas por día, ní un máximo de días para finalizar la grabación, el número de calendarios de rodaje no tiene un límite acotado, ya que se podría extender la producción indefinidamente sin llegar a completar nunca las grabaciones.

¿Cuantas posibilidades hay teniendo en cuenta

todas las restricciones.

A continuación de consideran las restricciones:

- **Número de actores:** 10
- **Número de tomas:** 30
  - En caso que no tener mas restricciones que este, las 30 tomas se podrían repartir en 30 días. El número de combinaciones ascendería a

$$30^{30} = 205891132094649000000000000000000000000000000$$

- **Límite de producción:** No es posible grabar más de 6 tomas por día.
  - Con esta nueva restricción podemos deducir que el menor tiempo posible. Eso resultaría en

$$30 \text{ tomas}/6 \text{ tomas por día} = 5 \text{ días}$$

- **Sincronización**: Los actores de deben de coincidir en las tomas en las que sus personajes aparecen juntos.
- **Optimización de costes**: Gasto por los servicios de los actores ha de ser el menor posible. Esto quiere decir, que cada día han de grabarse cuantas más tomas mejor (no más de 6).
- **Política de pago:** Cada actor cobra por el día que se desplaza al estudio, independiente del número de tomas a grabar.
- **Cálculo de posibilidades**
  - Teniendo en cuenta las restricciones anteriores, se debería de asignar a cada toma un día concreto, tratando de llenar siempre la cuata máxima de 6 tomas al día. Además, considerando que los actores ya están allí, entonces se podría reducir el problema a organizar las 30 tomas en 5 bloques de6.

$$A = 1,1,1,1,1,1,2,2,2,2,2,2,3,3,3,3,3,3,4,4,4,4,4,4,5,5,5,5,5,5$$

Las soluciones posibles para esta organización se calcularía mediante la fórmula de permutaciones con repetición sobre el conjunto A, lo que nos da un número total de combinaciones de:

$$P_{30}^{6,6,6,6,6} = \frac{30!}{6! \cdot 6! \cdot 6! \cdot 6! \cdot 6!} = \frac{30!}{(720)^5}$$

$$P = 1370874167589326400$$

Modelo para el espacio de soluciones

(*) ¿Cual es la estructura de datos que mejor se adapta

al problema? Argumentalo.(Es posible que hayas elegido una al principio y veas la necesidad de cambiar, arguentalo)

- Para representar esto de la mejor forma, la estructura de datos más adecuada es una lista de números enteros de tamaño 30.Cada posición n de la lista corresponde a la toma n de nuestra tabla de datos, y el número almacenado en esa posición representa el día que se le ha asignado.
- Siguiendo las restricciones mencionadas, cualquier lista que sea una solución válida será una permutación del conjunto A. De esta manera, nos aseguramos de que cada día (del 1 al 5) aparezca exactamente 6 veces en la lista, respetando el límite de capacidad y cubriendo todas las tomas.

Según el modelo para el espacio de soluciones

(*)¿Cual es la función objetivo?

- Se puede decir que la función objetivo es aquella que calcula el número de desplazamientos a la grabación de los actores. Buscamos contabilizar el número de vecesque aparece cada actor de doblaje a lo largo de todos los días de grabación.
- Para obtener la función hay que agrupar las tomas asignadas a cada día.
  - Cada toma se representa con dos valores (binario), donde 1 y 0 representan "asistir" y "no asistir" respectivamente.
  - Debido a que un actor ha de ir al estudio independientemente de si graba una toma **o** más de 1, entonces se ha de incluir la función lógica **OR**. Esta se puede representar con $|$.
  - Finalmente, deberíamos contar todos los actores que han ido ese día al estudio. Es decir, sumanos los actores. Sería equivalente a la función count().
  - Con esta información podemos decir que función objetivo sería algo como:

$$\sum_{j=1}^{M} count(T_{1j}|T_{2j}|\ldots|T_{kj})$$

donde: $j = \text{índice del día}$,
$M : \text{total de días de duración de la grabación}$,

$$T_{ij}$$

$$\text{donde } i = \text{el número de toma y } j = \text{el día asignado}$$

(*)¿Es un problema de maximización o minimización?

Es un problema de minimización. Se intenta reducir al máximo la cantidad de veces que los actores van el estudio, agrupando las tomas de forma que los repartos coincidan lo más posible, y de esta manera reducir costes.

Ejemplo

Vamos a usar un ejemplo para que se vea la idea. Imaginemos que tenemos 4 tomas y 2 días disponibles (máximo 2 tomas por día).

- **Solución 1**: [1,1,2,2] Agrupamos las tomas 1 y 2 el Día 1, y las tomas 3 y 4 el Día 2.
  - Día 1 (Tomas 1 y 2): Imaginemos que el OR lógico da 5 actores.
  - Día 2 (Tomas 3 y 4): Imaginemos que el OR lógico da 4 actores.
  - Total pagas: $5 + 4 = \mathbf{9}$
- **Solución 2**: [1,2,1,2] Agrupamos las tomas 1 y 3 el Día 1, y las tomas 2 y 4 el Día 2.
  - Día 1 (Tomas 1 y 3): Imaginemos que el OR lógico da 6 actores.
  - Día 2 (Tomas 2 y 4): Imaginemos que el OR lógico da 6 actores.
  - Total pagas: $6 + 6 = \mathbf{12}$
- **Conclusión:**
  - La Solución 1 vuelve a ser mejor (9 que pagamos contra 12). Se peude ver que al mezclar tomas que no comparten actores (Solución 2), aumentamos innecesariamente el número de personas que deben desplazarse al estudio cada día.

Para encontrar la mejor combinación de rodaje entre todas las posibilidades, primero necesitamos una herramienta matemática que nos diga exactamente cuánto cuesta cualquier plan que se nos ocurra. La función **calcular_coste_rodaje** de es nuestro motor de evaluación: toma un plan de trabajo, aplica la lógica de asistencia de los actores y nos devuelve el gasto total en dinero.

In [4]:
```
# Es función lo que hace es sumar cuánto nos va a costar el rodaje
# según cómo hayamos repartido las tomas en los días.
def calcular_coste_rodaje(plan_rodaje, matriz_asistencia, verbose=T
  try:
    # Primero comprobamos que hayamos asignado un día a cada toma
    assert len(plan_rodaje) == len(matriz_asistencia)
  except:
```

```
    print('CUIDADO: Las tomas y el plan de días no coinciden')
    return float('inf')

  # Hacemos una copia de los datos por seguridad
  datos_con_dia = matriz_asistencia.copy()
  # Añadimos una columna nueva para saber en qué día se graba cada
  datos_con_dia['DiaAsignado'] = pd.Series(plan_rodaje)

  total_pagas = 0
  # Vamos recorriendo cada día de rodaje
  for dia_actual in np.unique(plan_rodaje):
    # Seleccionamos solo las tomas que tocan ese día
    tomas_del_dia = datos_con_dia[datos_con_dia['DiaAsignado'] == d

    # tomas_del_dia.iloc[:,:-2] --> cogemos los 1s y 0s de los acto
    # np.any(, axis=0), --> Si un actor sale en alguna toma del día
    # 3. np.sum() --> Contamos cuántos actores tienen que venir, o
    actores_que_vienen = np.sum(np.any(tomas_del_dia.iloc[:,:-2], a
    total_pagas += actores_que_vienen

  if verbose: print(f'Total pagas con este plan {plan_rodaje}: {tot
  return total_pagas
```

Diseña un algoritmo para resolver el problema por fuerza bruta

La estrategia aquí es simple pero costosa: generamos todas las combinaciones posibles de tomas por día, calculamos el coste de cada una y nos quedamos con la más barata. Para esto, necesitamos crear una función que primero calcule todas las permutaciones y luego vaya probando una a una hasta dar con la solución óptima.

In [5]:
```
def fuerza_bruta(matriz_asistencia, max_tomas_dia):
  total_tomas = len(matriz_asistencia)
  # Calculamos el mínimo de días teóricos
  dias_minimos = math.ceil(total_tomas / max_tomas_dia)

  # Pintamos un poco de info inicial
  print(f'Actores totales en la grabación: {len(matriz_asistencia.l
  print(f'Tomas a grabar: {total_tomas} ')
  print(f'Límite de tomas al día: {max_tomas_dia} -> mínimo {dias_m

  # Creamos una lista base repartiendo las tomas entre los días mín
  # Ejemplo con 6 tomas y 3 por día -> [1, 1, 1, 2, 2, 2]
  plantilla_base = [dia for dia in range(1, dias_minimos + 1) for _
  plantilla_base = plantilla_base[:total_tomas]

  # Generamos TODAS las permutaciones (esto es lo que tarda tanto)
  todas_las_combinaciones = itertools.permutations(plantilla_base)
```

```
  coste_minimo = float('inf')
  mejor_plan = []

  # Vamos probando cada combinación una por una
  for plan in todas_las_combinaciones:
    # Calculamos cuánto cuesta este plan usando la función que hici
    coste_actual = calcular_coste_rodaje(plan, matriz_asistencia, v

    # Si este plan es más barato que el anterior más barato, nos lo
    if coste_actual < coste_minimo:
      coste_minimo = coste_actual
      mejor_plan = plan

  print(f'Mejor asignación: {mejor_plan}, con coste {coste_minimo}'
  return mejor_plan, coste_minimo
```

Calcula la complejidad del algoritmo por fuerza bruta

La complejidad es:

$$\frac{n!}{k!^M}$$

Este es principal inconveniente de este método, donde el número de combinaciones puede incrementar muy rápidadmente. Como esto crece a un ritmo de factorial $O(n!)$. Para nuestro caso de 30 tomas, no será posible probar todas las posibilidades, a no ser que lo hagamos por computación cuántica.

IMPORTANTE Debido a que la complejidad es factorial, no será posible obtener un resultado usando 30 registros. Para comprobar que la función trabaja correctamente, reduciré a 5 registros (tomas).

Aplica el algoritmo **Fuerza Bruta** al juego de datos **original reducido**

In [6]:
```
# Creamos una variable 'data_para_fuerza_bruta' con solo 5 tomas
# Para comprobar que el código funciona con menos registros.
# Seleccionamos las primeras 5 filas (iloc[0:5])
data_para_fuerza_bruta = data.iloc[0:5].copy()
```

In [7]:
```
# Ponemos el cronómetro
inicio_cronometro = time.time()

# Ejecutamos la función fuerza bruta
_, mejor_coste = fuerza_bruta(data_para_fuerza_bruta, 6)

# Calculamos la duración.
tiempo_total = round(time.time() - inicio_cronometro, 2)
print(f'Búsqueda Óptima: coste {mejor_coste} en {tiempo_total} segu
```

```
Actores totales en la grabación: 10
Tomas a grabar: 5
Límite de tomas al día: 6 -> mínimo 1 días

Mejor asignación: (1, 1, 1, 1, 1), con coste 7
Búsqueda Óptima: coste 7 en 0.37 segundos
```

Aplica el algoritmo **Fuerza Bruta** al juego de datos **aleatorios reducido** generado

In [8]:
```
# Creamos una variable 'data_para_fuerza_bruta' con solo 5 tomas
# Para comprobar que el código funciona con menos registros.
# Seleccionamos las primeras 5 filas (iloc[0:5])
datos_aleatorios_para_fuerza_fruta = df_aleatorio.iloc[0:5].copy()
```

In [9]:
```
# Ponemos el cronómetro
inicio_cronometro = time.time()

# Ejecutamos la función fuerza bruta
_, mejor_coste = fuerza_bruta(datos_aleatorios_para_fuerza_fruta, 6

# Calculamos la duración.
tiempo_total = round(time.time() - inicio_cronometro, 2)
print(f'Búsqueda Óptima: coste {mejor_coste} en {tiempo_total} segu
```

```
Actores totales en la grabación: 10
Tomas a grabar: 5
Límite de tomas al día: 6 -> mínimo 1 días

Mejor asignación: (1, 1, 1, 1, 1), con coste 10
Búsqueda Óptima: coste 10 en 0.29 segundos
```

(*)Diseña un algoritmo que mejore la complejidad del algortimo por fuerza bruta. Argumenta porque crees que mejora el algoritmo por fuerza bruta

Voraz

Tras comprobar que la fuerza bruta es inviable ($O(n!)$), una posible alternativa es implementar un algoritmo voraz. Esta técnica es ideal cuando buscamos eficiencia y el espacio de soluciones es muy grande.

¿Por qué elegir esta técnica?

- Según la definición de clase, la técnica voraz se basa en dividir el problema por etapas, tomando en cada una la decisión más ambiciosa y óptima sin mirar atrás.

- **Eficiencia**: Es la técnica más recomendable por su rapidez.
- **Aplicabilidad**: Cumple con las características necesarias: tenemos candidatos (tomas), una función de selección (agrupar por proximidad de actores), y una función de factibilidad (máximo k tomas por día).

Funcionamiento Aplicado al Problema:

- Selección: Localizamos la toma con más actores y asignamos el día.
- Proximidad: Calculamos cuántos actores comparten las demás tomas con la elegida.
- Ambición: Agrupamos las tomas con mayor proximidad para minimizar costes inmediatamente. Conclusión: Aunque voraz no garantiza el óptimo global en todos los casos, nos proporciona una solución factible en un timepo menor, superando el mayor inconveniente de la fuerza bruta.

In [10]:
```
# Este algoritmo construye la solución día a día, toma la mejor dec
def alg_voraz(matriz_asistencia, max_tomas_dia, verbose=True):

    # --- Complejidad en área 1: Preparación [O(n * m)] ---

    # Trabajamos sobre una copia para no romper los datos originale
    datos_trabajo = matriz_asistencia.copy()
    total_tomas = len(datos_trabajo)
    dias_totales = math.ceil(total_tomas / max_tomas_dia)

    if verbose:
        print(f'Actores totales en la grabación: {len(datos_trabajo
        print(f'Tomas a grabar: {total_tomas}')
        print(f'Límite de tomas al día: {max_tomas_dia} ->  mínimo

    # Array para guardar qué día le toca a cada toma
    plan_final = np.zeros(total_tomas)

    # --- Complejidad en área 2: Bucle principal [Se repite n/k vec
    # Vamos rellenando los días uno por uno
    for dia_actual in range(1, dias_totales + 1):
        scores_parecidos = np.zeros(len(datos_trabajo))

        # Buscamos la toma con más actores para que sea nuestra ref
        indice_toma_ancla = datos_trabajo['Total'].idxmax()
        plan_final[indice_toma_ancla] = dia_actual
        # Marcamos esta toma para no volver a usarla
        scores_parecidos[datos_trabajo.index.get_loc(indice_toma_an

        # --- Complejidad en área 3: Compatibilidad [O(n * m)] ---

        # Calculamos cuántos actores comparten las demás tomas con
        for posicion, indice_fila in enumerate(datos_trabajo.index)
            if indice_fila != indice_toma_ancla:
                # np.all comprueba qué actores coinciden en las tom
                scores_parecidos[posicion] = np.sum(np.all(datos_tr
```

```
        # ---Complejidad en área 4: Ordenación [O(n log n)] ---

        # Ordenamos las tomas por compatibilidad y cogemos las mejo
        datos_trabajo['Compatibilidad'] = scores_parecidos
        tomas_ordenadas = datos_trabajo.sort_values('Compatibilidad

        # Seleccionamos las k-1 tomas más parecidas para agruparlas
        tomas_a_agrupar = tomas_ordenadas[:min(max_tomas_dia - 1, l
        plan_final[tomas_a_agrupar] = dia_actual

        # ---Complejidad en área 5: Actualización [O(n)] --

        # Quitamos las tomas ya asignadas de la lista de pendientes
        datos_trabajo.drop(tomas_a_agrupar, axis=0, inplace=True)
        datos_trabajo.drop(indice_toma_ancla, axis=0, inplace=True)

    # ---Complejidad en área 6: Evaluación final [O(n * m)] ---

    # Convertimos a lista y calculamos el coste final
    plan_final = list(plan_final)
    coste_final = calcular_coste_rodaje(plan_final, matriz_asistenc

    if verbose: print(f'Solución voraz con coste {coste_final}: {pl
    return plan_final, coste_final
```

Aplica el algoritmo **Voraz** al juego de datos **original**

In [11]:
```
# Cronómetro
inicio_voraz = time.time()

# Ejecutamos la función voraz
_, coste_voraz = alg_voraz(data, 6, verbose=True)

# Calculamos y pintamos el tiempo y el resultado
tiempo_voraz = round(time.time() - inicio_voraz, 2)
print(f'Estrategia Voraz: coste {coste_voraz} en {tiempo_voraz} seg
```

```
Actores totales en la grabación: 10
Tomas a grabar: 30
Límite de tomas al día: 6 ->  mínimo 5 días

Solución voraz con coste 35: [np.float64(1.0), np.float64(3.0), np.f
loat64(2.0), np.float64(2.0), np.float64(2.0), np.float64(1.0), np.f
loat64(1.0), np.float64(3.0), np.float64(2.0), np.float64(2.0), np.f
loat64(1.0), np.float64(1.0), np.float64(3.0), np.float64(4.0), np.f
loat64(2.0), np.float64(5.0), np.float64(3.0), np.float64(5.0), np.f
loat64(4.0), np.float64(1.0), np.float64(5.0), np.float64(3.0), np.f
loat64(4.0), np.float64(5.0), np.float64(3.0), np.float64(4.0), np.f
loat64(5.0), np.float64(4.0), np.float64(4.0), np.float64(5.0)]
Estrategia Voraz: coste 35 en 0.21 segundos
```

Aplica el algoritmo **Voraz** al juego de datos **aleatorios** generado

In [12]:
```
# Cronómetro
inicio_voraz = time.time()

# Ejecutamos la función voraz
_, coste_voraz = alg_voraz(df_aleatorio, 6, verbose=True)

# Calculamos y pintamos el tiempo y el resultado
tiempo_voraz = round(time.time() - inicio_voraz, 2)
print(f'Estrategia Voraz: coste {coste_voraz} en {tiempo_voraz} seg
```

```
Actores totales en la grabación: 10
Tomas a grabar: 30
Límite de tomas al día: 6 ->  mínimo 5 días

Solución voraz con coste 44: [np.float64(1.0), np.float64(2.0), np.f
loat64(4.0), np.float64(5.0), np.float64(3.0), np.float64(4.0), np.f
loat64(4.0), np.float64(4.0), np.float64(2.0), np.float64(2.0), np.f
loat64(1.0), np.float64(4.0), np.float64(2.0), np.float64(5.0), np.f
loat64(3.0), np.float64(4.0), np.float64(5.0), np.float64(5.0), np.f
loat64(1.0), np.float64(1.0), np.float64(5.0), np.float64(1.0), np.f
loat64(5.0), np.float64(3.0), np.float64(2.0), np.float64(3.0), np.f
loat64(3.0), np.float64(1.0), np.float64(2.0), np.float64(3.0)]
Estrategia Voraz: coste 44 en 0.2 segundos
```

(*)Calcula la complejidad del algoritmo

Variables:

- n: Número total de tomas.
- m: Número de actores (columnas).
- k: Máximo de tomas por día

Análisis por secciones:

- **Preparación:** $O(n \cdot m)$ (Copia de la matriz).
- **Bucle Exterior (Días):** Se ejecuta exactamente $\lceil n/k \rceil$ veces.
- **Dentro del bucle (por cada día):**
  - **Búsqueda del Ancla:** $O(n)$ (Encontrar el máximo en la columna 'Total').
  - **Cálculo de Compatibilidad:** $O(n \cdot m)$ (Un bucle que recorre n tomas y compara m actores con np.all).
  - **Ordenación (sort_values):** $O(n \log n)$ (Algoritmo Timsort de Pandas sobre las tomas restantes).
  - **Actualización (drop):** $O(n)$ (Eliminar filas del DataFrame). La complejidad es:

$$T(n) = O(n \cdot m) + \frac{n}{k} \cdot [\underbrace{n \cdot m}_{\text{Compatibilidad}} + \underbrace{n \log n}_{\text{Ordenación}} + \underbrace{2n}_{\text{Ancla y Drop}}]$$

Si distribuimos el término $\frac{n}{k}$:

$$T(n) = O(n \cdot m) + O\left(\frac{n^2 \cdot m}{k}\right) + O\left(\frac{n^2 \log n}{k}\right) + O\left(\frac{2n^2}{k}\right)$$

Resultado Final (Big O):

$$O\left(\frac{n^2 \cdot (m + \log n)}{k}\right)$$

y como la compatibilidad de actores ($n^2 \cdot m$) pesa más que la ordenación ($n^2 \log n$).

$$\frac{n}{k} \times (n \times m) = \frac{n^2 \cdot m}{k}$$

Si consideramos m como n, entonces podemos decir que el término dominante $\frac{n^3}{k}$: Representa el bucle principal donde recorremos los días y, dentro de cada día, evaluamos la compatibilidad de las tomas. En este caso hablamos de $O(n^3)$.

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

Búsqueda Aleatoria

Ya vimos que la Fuerza Bruta es inviable aquí, y por otra parte la voracidad no asegura la solución más óptima. Ya que el enunciado pide "diseñar un juego de datos de entrada aleatorios", hemos llegado a la conclusión de que usar una exploración aleatoria podría ser una pista en este caso, por lo tanto vamos a proceder a aplicar el algoritmo de búsqueda aleatoria.

La lógica a seguir será: en lugar buscar todas las combinaciones, lanzamos X intentos al azar y nos quedamos con el mejor de ellos. Al ser probabilístico y heurístico, no podemos asegurar tampoco que es la solución más óptima.

In [13]:
```
def busqueda_aleatoria(intentos, data, max_tomas_dia, mostrar_info=
    # Calculamos en número de tomas
    total_tomas = len(data)
    # Calculamos días necesarios y redondeamos hacia arriba
    dias_necesarios = math.ceil(total_tomas / max_tomas_dia)

    if mostrar_info:
        # data.shape[1] nos da el numero de columnas (actores)
        print(f'Actores totales en la grabación: {data.shape[1]} ')
```

```
        print(f'Tomas a grabar: {total_tomas} ')
        print(f'Límite de tomas al día: {max_tomas_dia} -> Mínimo {

    # --- Complejidad en área 1: Preparación [O(n)] ---

    # Crearemos una lista con los días del tipo [1,1,1, 2,2,2, ...]
    # Lista vacía para empezar
    distribucion_base = []

    # Se recorre cada día necesario
    for dia in range(1, dias_necesarios + 1):

        # Se añade el número del día actual "max_tomas_dia" veces
        for _ in range(max_tomas_dia):
            distribucion_base.append(dia)

    # Ajustamos al tamaño real de tomas por si acaso
    distribucion_base = distribucion_base[:total_tomas]

    # Hacemos un "for" de exploración
    coste_minimo = float('inf')
    mejor_plan = []

    # --- Complejidad en área 2: Bucle exploración [intentos (i)] -

    for i in range(intentos):

        # --- Complejidad en área 3: Mezcla aleatoria [O(n)] ---

        # Desordenamos los días para probar una mezcla nueva
        random.shuffle(distribucion_base)

        # --- Complejidad en área 4: coste de rodaje [O(n)] ---

        # Calculamos el coste del shuffle actual
        coste_actual = calcular_coste_rodaje(distribucion_base, dat

        # Si es mejor que lo anterior lo guardamos
        if coste_actual < coste_minimo:
            coste_minimo = coste_actual

            # --- Complejidad en área 5: copia de solución [O(n)] -
            mejor_plan = distribucion_base.copy()
            if mostrar_info:
                print(f'Intento {i+1}: Nuevo mejor coste {coste_min

    if mostrar_info:
        print(f'\nFinalizado. Coste final: {coste_minimo}')

    return mejor_plan, coste_minimo
```

Aplica el algoritmo **Búsqueda Aleatoria** al juego de datos **original**

In [14]:
```
# Cronómetro
```

```
inicio_aleatoria = time.time()

# Ehecutamos la función y usamos 10000 intentos como ejemplo
plan_aleatorio, coste_aleatorio = busqueda_aleatoria(1000, data, ma

# --- RESULTADOS ---
tiempo_aleatoria = round(time.time() - inicio_aleatoria, 2)
print(f'Estrategia Aleatoria: coste {coste_aleatorio} en {tiempo_al
```

```
Actores totales en la grabación: 11
Tomas a grabar: 30
Límite de tomas al día: 6 -> Mínimo 5 días

Intento 1: Nuevo mejor coste 40
Intento 3: Nuevo mejor coste 38
Intento 6: Nuevo mejor coste 36
Intento 99: Nuevo mejor coste 35
Intento 445: Nuevo mejor coste 34

Finalizado. Coste final: 34
Estrategia Aleatoria: coste 34 en 9.69 segundos
```

Aplica el algoritmo **Búsqueda Aleatoria** al juego de datos **aleatorios** generado

In [15]:
```
# Cronómetro
inicio_aleatoria = time.time()

# Ejecutamos la función y usamos 1000 intentos como ejemplo
plan_aleatorio, coste_aleatorio = busqueda_aleatoria(1000, df_aleat

# --- RESULTADOS ---
tiempo_aleatoria = round(time.time() - inicio_aleatoria, 2)
print(f'Estrategia Aleatoria: coste {coste_aleatorio} en {tiempo_al
```

```
Actores totales en la grabación: 11
Tomas a grabar: 30
Límite de tomas al día: 6 -> Mínimo 5 días

Intento 1: Nuevo mejor coste 50
Intento 2: Nuevo mejor coste 47
Intento 13: Nuevo mejor coste 46
Intento 640: Nuevo mejor coste 45

Finalizado. Coste final: 45
Estrategia Aleatoria: coste 45 en 11.09 segundos
```

Variables:

- n: Número total de tomas.
- m: Número de actores (columnas).
- i: Número de intentos (iteraciones de búsqueda)

Análisis por secciones:

- **Preparación:** $O(n)$ (Crear la lista base y reordenarla).
- **Bucle exploración:** Se ejecuta i veces.
- **Dentro del bucle (por cada intento):**
  - **Mezcla:** $O(n)$ (se reordena la lista de tomas)
  - **Evaluación:** $O(n\dot{m})$ (Recorre las tomas y actores para calcular el coste).
  - **Actualización:** $O(n)$ (Copia el mejor plan encontrando hasta el momento).

La complejidad es:

$$T(n) = O(n) + i \cdot [\underbrace{n}_{\text{Shuffle}} + \underbrace{n \cdot m}_{\text{Evaluación}} + \underbrace{n}_{\text{Copia}}]$$

Si distribuimos el término i:

$$T(n) = O(n) + O(i \cdot n) + O(i \cdot n \cdot m) + O(i \cdot n)$$

Resultado Final (Big O):

$$\boxed{O(i \cdot n \cdot m)}$$

Esta complejidad es lineal respecto al número de intentos (i) y al número de tomas (n), multiplicada por la complejidad de calcular el coste de cada rodaje.La fórmula aproximada es: $O(i \cdot n \cdot m)$. Entonces si $n = m$ podríamos decir que la complejidad total es $O(i \times n^2)$

Enumera las referencias que has utilizado(si ha sido necesario) para llevar a cabo el trabajo

- Reyero Díez, R. (2024). Algoritmos de optimización. Universidad Internacional de Valencia.
- Universidad Internacional de Valencia. (2024). Guía de Actividades: Actividades Guiadas y Trabajo Práctico de Algoritmos de Optimización
- Numpy: https://numpy.org/doc/
- Time: https://docs.python.org/3/library/time.html
- Pandas: https://pandas.pydata.org/docs/

Describe brevemente las lineas de como crees que es posible avanzar en el estudio del

problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

Propuestas de mejora para el sistema de doblaje

Para dar el siguiente paso en este proyecto, en caso de que el número de tomas tomas o las restricciones aumente considerablemnte, entonces téndremos que mirar otros algoritmos.

Metaheurísticas

Ahora mismo, si un algoritmo encuentra una buena solución, entonces ya no continua. Por lo que necesitamos técnicas que **no** se conformen con lo primero que encuentran:

- **Búsqueda Tabú**: Básicamente, le damos al algoritmo un 'diario' para que recuerde qué soluciones ya probó y no vuelva a caer en los mismos errores (o en los mismos 'mínimos locales').
- **Recocido Simulado**: Al principio le dejamos probar todo tipo de soluciones (donde puede incluso empeorar) para explorar todo el terreno, y poco a poco hacemos que se enfoque en la mejor opción encontrada.

Algoritmos Poblacionales

Si el problema crece demasiado, sería mejor probar muchas veces a la vez, en vez de buscar una solución. Es mejor probar muchas a la vez:

- **Algoritmos Genéticos**: Creamos un grupo de posibles horarios. Hacemos que los mejores horarios combinen sus partes buenas y metemos cambios aleatorios para ver si encontramos algo mejor.
- **Colonia de Hormigas**: Vamos probando probando combinaciones y dejamos pistas las agrupaciones de tomas que resultan ser más baratas.

Hibridación

Otra opción sería no usar una sola técnica, sino mezclar las partes de buena de cada una:

- Podemos usar nuestro **algoritmo voraz** actual para tener una solución decente rápida, y luego aplicamos **GRASP** (aleatoriedad) para encontrar la verdadera mejor opción.

Modificarlo para casos reales o nuevos modelos.

El modelo actual para alguién experto, puede ser considerado simple. Y puedes que no se capaz de adaptarse a la vida real. A veces esta adaptación

a la vida real podría llevar a caminos más complejos pero efectivos:

- **Varios o diferentes objetivos**: En este caso importa el dinero, pero en otro podría ser el terminar el proyecto lo antes posible.
- **Cambios de última hora**: El algoritmo debería poder recalcular todo si un actor se enferma de repente (penasando en un modelo más dinámico).

Escalabilidad

Por último, habría que tener en cuenta la duración de esos algoritmos nuevos. No sirve de nada una solución perfecta si tarda tres días en calcularse.

Clasificación Integral de Algoritmos de Optimización

Esta tabla la creamos para tener un visión general de los diferentes algoritmos. Esta nos ha ayudado a a decidir que tipos de algoritmos podrían haberse utilizado en el proyecto.

Algoritmo	Categoría	Forma de Calcular	Calidad de Solución	Características Principales
Simplex	Programación Matemática	Determinista	Exacto	Se desplaza por la frontera de la región factible hasta encontrar el óptimo matemático.
Ramificación y Poda	Búsqueda Informada	Determinista	Exacto	Divide el problema en subproblemas y descarta ramas que no pueden superar al mejor resultado hallado.
Programación Dinámica	Programación Dinámica	Determinista	Exacto	Divide el problema en subproblemas superpuestos y almacena sus soluciones en tablas para evitar cálculos repetidos.
Vuelta Atrás (Backtracking)	Búsqueda Sistemática	Determinista	Exacto	Explora un árbol de expansión de manera exhaustiva y retrocede cuando una rama deja de ser prometedora.
Ordenación (p.ej. Merge	Algoritmos de	Determinista	Exacto	Sigue pasos finitos y precisos para organizar una lista

Sort)	Ordenación			bajo un criterio estricto.
Algoritmos Voraces (Greedy)	Técnica de Diseño	Determinista	Heurístico / Aproximado	Elige en cada etapa la decisión óptima local sin considerar las consecuencias futuras; no garantiza el óptimo global.
Multiarranque (Multi-start)	Metaheurística Constructiva	Probabilista / Determinista	Heurístico	Repite el proceso de generar una solución inicial y aplicar búsqueda local para explorar diversas regiones y evitar mínimos locales.
Búsqueda Aleatoria	Metaheurística	Probabilista	Heurístico	Genera soluciones totalmente al azar en cada iteración y devuelve la mejor encontrada.
Búsqueda Local	Metaheurística	Determinista / Probabilista	Heurístico	Parte de una solución inicial y busca mejorarla explorando su vecindad inmediata; tiene riesgo de estancarse en óptimos locales.
GRASP	Metaheurística	Probabilista	Heurístico	Combina una fase constructiva voraz-aleatoria con una fase de mejora mediante búsqueda local.
Búsqueda Tabú	Metaheurística	Determinista / Probabilista	Heurístico	Utiliza una memoria adaptativa (lista tabú) para prohibir movimientos recientes y forzar la exploración de nuevas áreas.
Recocido Simulado	Metaheurística	Probabilista	Heurístico	Acepta soluciones peores con una probabilidad que disminuye con el tiempo, imitando el enfriamiento de metales.

Algoritmos Genéticos	Poblacional	Probabilista	Heurístico	Se basan en mecanismos de selección natural, cruce y mutación aplicados a una población de soluciones.
Colonia de Hormigas (ACO)	Poblacional	Probabilista	Heurístico	Múltiples agentes depositan feromonas en las rutas para guiar probabilísticamente hacia la mejor solución compartida.
Descenso del Gradiente	Optimización Continua	Determinista	Exacto (en funciones convexas)	Algoritmo iterativo que minimiza una función continua siguiendo la dirección opuesta al gradiente.
Dijkstra	Búsqueda en Grafos	Determinista	Exacto	Encuentra el camino más corto desde un nodo origen al resto, siempre que no haya pesos negativos.